

Model-Theoretic Quantifier Elimination

A Lean formalization over `mathlib4`

Yağız Kaan Aydoğdu, Yusuf Demir, Salih Erdem Koçak

Advisors: Ayhan Günaydın, Metin Ersin Arıcan

Contents

1. Model Theory
2. Type Theory
3. Formalization in Lean

Model Theory

What is Model Theory?

- Model theory is a branch of mathematical logic where we study mathematical structures by considering the first-order sentences true in those structures and the sets definable by first-order formulas.
- Given concrete mathematical structures, we can obtain new information about the structure using model-theoretic techniques.

Language

- In mathematical logic, we use first-order languages to describe mathematical structures.
- A *language* $\mathcal{L}$ consists of:
 - a set of function symbols $\mathcal{F}$
 - a set of relation symbols $\mathcal{R}$
 - a set of constant symbols $\mathcal{C}$
- Examples:
 - The language of rings $\mathcal{L}_r = \{+, -, \cdot, 0, 1\}$
 - The language of orders $\mathcal{L}_o = \{<\}$

Structure

- A structure interprets the symbols of a language on a domain.
- Formally, an $\mathcal{L}$ -structure $\mathcal{M}$ consists of:
 - a non-empty set M (the domain/universe),
 - for each function symbol $f \in \mathcal{F}$, a function $f^{\mathcal{M}} : \mathcal{M}^n \rightarrow \mathcal{M}$,
 - for each relation symbol $R \in \mathcal{R}$, a relation $R^{\mathcal{M}} \subseteq \mathcal{M}^n$,
 - for each constant symbol $c \in \mathcal{C}$, an element $c^{\mathcal{M}} \in \mathcal{M}$.
- Examples: $(\mathbb{N}, 0, 1, +, \cdot)$, $(\mathbb{Q}, \leq)$.

Terms

- An $\mathcal{L}$ -term is:
 - a constant symbol c ,
 - a variable v_i
 - or a function symbol f applied to terms $t_1, \dots, t_n$.

Formula and Sentence

- An *atomic $\mathcal{L}$ -formula* is a relation symbol applied to terms, or an equality between terms.
- *$\mathcal{L}$ -formulas* are built from atomic formulas using logical connectives and quantifiers.
- An *$\mathcal{L}$ -sentence* is a formula with no free variables.
- Example: $\exists z (x < z \wedge z < y)$ is formula but not a sentence, as it has free variables x and y .

Satisfaction

- We say that an $\mathcal{M}$ *satisfies* a formula φ , written $\mathcal{M} \models \varphi$, if φ holds in $\mathcal{M}$.
- Formally, the satisfaction relation is defined inductively on the structure of formulas.
- Example: $(\mathbb{Q}, <) \models \forall x \forall y (x < y \implies \exists z x < z \wedge z < y)$.

Theory and Model

- An $\mathcal{L}$ -theory is a set of $\mathcal{L}$ -sentences.
- We say that a structure $\mathcal{M}$ is a *model* of a theory T , written $\mathcal{M} \models T$, if $\mathcal{M}$ satisfies every sentence in T .
- A theory is *satisfiable* if it has a model.
- Moreover, we say that a theory T satisfies a sentence φ , written $T \models \varphi$, if every model of T satisfies φ .

Quantifier Elimination

- A theory T has *quantifier elimination* if for every formula φ , there is a quantifier-free formula ψ such that

$$T \models \varphi \leftrightarrow \psi$$

Dense Linear Orders without Endpoints

- Let $\mathcal{L}_o = \{<\}$ be the language of orders.
- Let DLO be the $\mathcal{L}_o$ -theory consisting of the following sentences:

$\forall x \neg(x < x)$	(Irreflexivity)
$\forall x \forall y \forall z (x < y \wedge y < z \rightarrow x < z)$	(Transitivity)
$\forall x \forall y (x < y \vee x = y \vee y < x)$	(Linearity)
$\forall x \forall y (x < y \rightarrow \exists z (x < z \wedge z < y))$	(Density)
$\forall x \exists y \exists z (y < x \wedge x < z)$	(No endpoints)

- Examples of models of DLO: $(\mathbb{Q}, <)$, $(\mathbb{R}, <)$, $((0, 1), <)$ with the usual order.

Does DLO have Quantifier Elimination?

- DLO has quantifier elimination.
- Every formula in the language $\{<\}$ is equivalent over DLO to a quantifier-free formula.
- Examples:

$$\exists z (x < z \wedge z < y) \iff x < y$$

$$\exists z (x < z \wedge w < z \wedge z < y) \iff x < y \wedge w < y$$

$$\exists z (x < z \wedge z < y \wedge \neg(z < w)) \iff x < y \wedge (w < y \vee w = y)$$

Type Theory

Judgements

- $a : A$ is a judgement: a has type A .
- Every expression has a type.
- Definitional equality is written $a \equiv b$.

Propositions as Types

- A proposition is viewed as a type of evidence.
- A proof of P is a term $p : P$.
- To prove a theorem is to construct an object of the corresponding type.

Constructive Reasoning

- A proof of $\exists x, P(x)$ gives a witness and evidence.
- A proof of $P \vee Q$ chooses one side and proves it.
- The law of excluded middle, $P \vee \neg P$, is not built in constructively.
- Lean allows classical reasoning when explicitly invoked.

Curry–Howard Isomorphism

$$\llbracket P \Rightarrow Q \rrbracket \equiv \llbracket P \rrbracket \rightarrow \llbracket Q \rrbracket$$

$$\llbracket P \wedge Q \rrbracket \equiv \llbracket P \rrbracket \times \llbracket Q \rrbracket$$

$$\llbracket \text{True} \rrbracket \equiv 1$$

$$\llbracket P \vee Q \rrbracket \equiv \llbracket P \rrbracket + \llbracket Q \rrbracket$$

$$\llbracket \text{False} \rrbracket \equiv 0$$

$$\llbracket \forall x : A. P \rrbracket \equiv \prod x : A. \llbracket P \rrbracket$$

$$\llbracket \exists x : A. P \rrbracket \equiv \sum x : A. \llbracket P \rrbracket$$

Formalization in Lean

```

/-- A formula is equivalent to a quantifier-free formula over a theory. -/
def IsQFEquivalent (T : L.Theory) {α : Type*} (φ : L.Formula α) : Prop :=
  ∃ ψ : L.Formula α, ψ.IsQF ∧ φ ⇔[T] ψ

/-- A theory has quantifier elimination if every formula is equivalent,
over the theory, to a quantifier-free formula. -/
def HasQuantifierElimination (T : L.Theory) : Prop :=
  ∀ {α : Type} (φ : L.Formula α), T.IsQFEquivalent φ

```

```
-- The theory of dense linear orders without endpoints
has quantifier elimination. -/
theorem dlo_hasQuantifierElimination :
  Language.order.dlo.HasQuantifierElimination := by
  ...
```

Thank you for listening!



Scan the QR code to checkout our PR in `mathlib4` Github repository!